2. Descripción del Problema

2.1 Situación actual

Actualmente, Konys Pizza maneja las reservas de sus clientes de manera completamente manual. Los clientes llaman por teléfono, escriben por WhatsApp o se presentan directamente en el local para separar una mesa. Este proceso presenta múltiples deficiencias que afectan tanto la operación del negocio como la experiencia del cliente.

2.2 Problemas identificados

- Doble reserva de mesas: cuando dos clientes reservan la misma mesa en el mismo horario sin que nadie lo note.
- Falta de visibilidad: el cliente no puede saber desde su casa qué mesas y horarios están disponibles.
- Carga operativa: el administrador debe gestionar manualmente cada reserva, lo que consume tiempo valioso en horas de alta demanda.
- Pérdida de clientes: quienes intentan comunicarse fuera del horario de atención, o cuando la línea está ocupada, no pueden reservar.
- Sin historial: no existe registro digital de las reservas pasadas ni de la demanda por día y horario.

> 📌 **Ejemplo real del problema**
> Imagina que Juan escribe a las 3 p. m. pidiendo una mesa para cuatro personas el sábado a las 7:00 p. m. El administrador lo anota en una libreta. Luego María llama y también pide esa mesa para el sábado a las 7:00 p. m., y el administrador, ocupado con los pedidos a domicilio, la agenda también. El resultado: dos grupos en la puerta, un cliente molesto y una mala reseña en Google.

2.3 Impacto del problema

El impacto de esta situación se puede medir en tres dimensiones principales:

- Impacto económico: pérdida de ingresos por mesas vacías, reservas dobles o clientes que no logran reservar.
- Impacto en la reputación: una mala experiencia reduce las recomendaciones y la calificación del restaurante en Google.
- Impacto operativo: el administrador invierte tiempo en coordinación manual en lugar de en la gestión del negocio.

4. Alcance del Proyecto

4.1 Lo que SÍ incluye el proyecto

- Módulo de cliente: visualización de mesas, selección de mesa, elección de fecha/hora y confirmación de la reserva.
- Módulo de administrador: registro y gestión de mesas, definición de horarios y visualización de reservas.
- Consulta de la carta digital (pizzas, entradas, postres y bebidas) en modo lectura.
- Interfaz de disponibilidad: visualización clara de franjas libres y ocupadas.
- Almacenamiento de datos con Supabase.
- Mockups de baja fidelidad de las pantallas principales de la app.
- Distribución de la app mediante Expo Go o un APK generado con EAS Build, con código en GitHub (mínimo 5 commits).
- Video explicativo del proyecto.
- Aplicación móvil multiplataforma (Android e iOS) con React Native y Expo, adaptable a diferentes tamaños de pantalla.

4.2 Lo que NO incluye el proyecto (fuera del alcance)

- Sistema de pagos en línea o anticipos de reserva.
- Pedidos a domicilio en línea (se mantiene el canal actual por WhatsApp).
- Versión web de la aplicación.
- Publicación en Google Play Store o App Store.
- Autenticación de clientes (registro de usuario).
- Notificaciones push, por correo electrónico, SMS o WhatsApp.
- Reportes o estadísticas de ventas.
- Gestión de inventario o de insumos de cocina.

⚠️ **Por qué es importante definir el alcance**
Definir claramente qué está dentro y qué está fuera del alcance evita el fenómeno conocido como 'scope creep' (expansión descontrolada del alcance). En proyectos reales, no delimitar el alcance es una de las principales causas de retrasos y sobrecostos. Las funcionalidades fuera del alcance pueden considerarse mejoras futuras.

3. Objetivos del Sistema

3.1 Objetivo general

Desarrollar una aplicación móvil multiplataforma (Android e iOS) con React Native y Expo que permita a los clientes de Konys Pizza reservar mesas de forma autónoma, visualizando la disponibilidad en tiempo real, y que brinde al administrador herramientas para gestionar las mesas, los horarios y las reservas de manera eficiente.

3.2 Objetivos específicos

1. Implementar una interfaz móvil intuitiva donde el cliente pueda ver las mesas del restaurante y seleccionar una para su reserva.
2. Permitir que el cliente elija fecha, hora y número de personas, y confirme su reserva sin necesidad de llamar o desplazarse.
3. Mostrar la disponibilidad de cada mesa (libre u ocupada) por franjas horarias.
4. Proveer al administrador un panel para registrar mesas, definir los horarios de atención y consultar las reservas.
5. Persistir todos los datos en una base de datos en Supabase para garantizar la disponibilidad de la información.
6. Compilar y distribuir la aplicación con Expo (Expo Go y APK generado con EAS Build), con el código fuente gestionado en un repositorio de GitHub (mínimo 5 commits).
7. Documentar el diseño mediante mockups de baja fidelidad y presentar el trabajo en un video explicativo.

> 💡 **Consejo para el estudiante**
> Los objetivos específicos son la columna vertebral del proyecto. Cada objetivo debe traducirse en al menos un requisito funcional. Si un objetivo no se puede medir o validar, no está bien redactado. Usa verbos de acción: 'implementar', 'permitir', 'mostrar', 'proveer'.

5. Identificación de Actores

Un actor es cualquier persona, sistema o entidad que interactúa con la aplicación. Identificarlos correctamente permite definir qué puede hacer cada uno y qué permisos necesita.

Actor	Descripción y rol
Cliente	Persona que desea reservar una mesa en Konys Pizza. Accede al módulo de reservas sin necesidad de autenticarse. Puede ver la disponibilidad, consultar la carta y confirmar su reserva.
Administrador	Persona encargada de la gestión operativa del restaurante. Accede a un panel de administración para registrar mesas, definir horarios, ver reservas y administrar la carta. Se autentica con credenciales.
Sistema (Supabase)	Actor externo que actúa como base de datos. Recibe y responde consultas de la aplicación para leer y escribir datos de mesas, horarios, reservas y platos.

> 💡 **Nota**
> En este proyecto no se implementa un módulo de autenticación complejo para el cliente. El administrador puede proteger su panel con un PIN o una clave simple dentro de la app como medida básica de seguridad, aunque en producción se recomendaría una autenticación real.

ç

6. Requisitos Funcionales

Los requisitos funcionales describen qué debe hacer el sistema. Se organizan por módulo y se numeran para facilitar su trazabilidad.

> 📌 **¿Qué es la trazabilidad?**
> Trazabilidad significa que cada requisito puede rastrearse desde su origen (necesidad del negocio) hasta su implementación (código) y prueba (caso de prueba). Es una buena práctica en ingeniería de software identificar cada requisito con un código único como RF-01.

6.1 Módulo Cliente

RF-01: Visualización de mesas disponibles

El sistema deberá mostrar en la pantalla de inicio una representación visual en tarjetas de las mesas del restaurante. Cada mesa mostrará su nombre, la zona (terraza o salón), su capacidad y su estado general. El cliente podrá ver de un vistazo cuántas mesas existen y cuáles tienen cupo.

RF-02: Selección de mesa

El cliente podrá tocar una mesa disponible para seleccionarla. Al seleccionarla, el sistema mostrará los detalles de esa mesa y los horarios disponibles para la fecha seleccionada.

RF-03: Elección de fecha

El sistema deberá ofrecer un calendario o selector de fecha que permita al cliente elegir el día de su reserva. El sistema solo mostrará días habilitados dentro del horario de operación del restaurante.

RF-04: Visualización de horarios disponibles

Una vez seleccionadas la mesa y la fecha, el sistema deberá mostrar los bloques de tiempo (slots) disponibles y no disponibles para ese día y mesa. Los slots ocupados estarán visualmente diferenciados de los disponibles (ej: gris para ocupado, blanco o verde para libre).

RF-05: Reserva de mesa

El cliente podrá seleccionar un slot disponible y confirmar su reserva. El sistema deberá solicitar el nombre del cliente, el número de personas (que no podrá superar la capacidad de la mesa) y opcionalmente un número de contacto. Al confirmar, el slot pasará a estado 'ocupado' y la reserva quedará registrada en la base de datos.

RF-06: Confirmación de reserva

Tras completar la reserva, el sistema mostrará en pantalla un mensaje de confirmación con los datos: mesa, fecha, hora, número de personas y nombre del cliente. Este mensaje servirá como comprobante visual de la reserva.

RF-07: Consulta de la carta digital

El cliente podrá consultar la carta del restaurante organizada por categorías (pizzas, entradas, postres y bebidas), con nombre, descripción y precio de cada plato. Los platos marcados como no disponibles no se mostrarán.

6.2 Módulo Administrador

RF-08: Acceso al panel de administración

El sistema deberá contar con una sección de administración accesible desde la pestaña 'Admin' de la app. El acceso deberá estar protegido por un PIN o una contraseña simple.

RF-09: Registro de mesas

El administrador podrá registrar nuevas mesas ingresando: nombre o número, zona (terraza o salón), capacidad de personas y, opcionalmente, una foto. Cada mesa quedará disponible para recibir reservas.

RF-10: Listado de mesas

El sistema mostrará una lista de todas las mesas registradas con su información básica: nombre, zona, capacidad y estado actual (activa / inactiva). El administrador podrá ver este listado desde el panel de administración.

RF-11: Gestión de horarios

El administrador podrá definir o modificar el horario de atención por día de la semana y la duración de cada reserva. Por ejemplo, indicar que la pizzería atiende en dos jornadas: de 12:30 p. m. a 2:30 p. m. y de 5:00 p. m. a 11:00 p. m., con reservas de 60 minutos. El sistema usará esta información para generar los slots disponibles.

RF-12: Visualización de reservas

El administrador podrá consultar todas las reservas registradas, con la posibilidad de filtrar por fecha o por mesa. Cada reserva mostrará: nombre del cliente, mesa, fecha, hora, número de personas y estado.

RF-13: Cambio de estado de mesa

El administrador podrá marcar una mesa como 'activa' o 'inactiva'. Esta funcionalidad sirve para casos como mantenimiento, eventos privados o cierre temporal de la terraza. Cuando una mesa está inactiva, sus slots no aparecerán en el módulo de cliente.

RF-14: Gestión de la carta

El administrador podrá agregar, editar, ocultar y eliminar platos de la carta digital, indicando nombre, categoría, descripción, precio y disponibilidad.

7. Requisitos No Funcionales

Los requisitos no funcionales describen cómo debe comportarse el sistema, no qué debe hacer. Incluyen aspectos de rendimiento, seguridad, usabilidad y mantenibilidad.

Requisito	Descripción
RNF-01 · Usabilidad	La interfaz debe ser intuitiva y no requerir capacitación. Un cliente sin experiencia técnica debe poder reservar una mesa en menos de 3 pasos sin necesidad de instrucciones adicionales.
RNF-02 · Adaptabilidad	La aplicación debe adaptarse correctamente a distintos tamaños de pantalla de celulares y tablets, en orientación vertical, y comportarse de forma consistente en Android e iOS.
RNF-03 · Rendimiento	Las pantallas deben cargarse en menos de 3 segundos bajo condiciones normales de red. Las consultas a Supabase deben responder en menos de 1 segundo para una buena experiencia de usuario.
RNF-04 · Disponibilidad	La aplicación debe estar disponible el 99% del tiempo. Al usar Supabase y Expo como infraestructura, ambos ofrecen alta disponibilidad dentro de sus planes gratuitos.
RNF-05 · Seguridad	Los datos de los clientes (nombre y contacto) deben transmitirse de forma segura mediante HTTPS. El acceso al panel de administración debe estar protegido para evitar modificaciones no autorizadas.
RNF-06 · Mantenibilidad	El código debe estar organizado en carpetas y archivos separados por funcionalidad (pantallas, componentes y servicios) con comentarios claros. Esto facilita que otro desarrollador pueda entender y modificar el sistema en el futuro.
RNF-07 · Compatibilidad	La aplicación debe funcionar correctamente en Android e iOS, en las versiones mínimas soportadas por el SDK de Expo utilizado, tanto en Expo Go como en el APK instalado.
RNF-08 · Escalabilidad	La arquitectura debe permitir agregar nuevas mesas, zonas o platos sin necesidad de reestructurar la base de datos o refactorizar el código de manera significativa.
RNF-09 · Manejo de conexión	La app debe mostrar un mensaje claro cuando no haya conexión a internet o falle una consulta a Supabase, en lugar de cerrarse o quedarse en blanco.

8. Casos de Uso

Un caso de uso describe la interacción entre un actor y el sistema para lograr un objetivo específico. A continuación, se presentan los casos de uso principales del sistema.

8.1 Resumen de casos de uso

Código	Caso de uso	Actor	Descripción breve
CU-01	Visualizar disponibilidad	Cliente	El cliente accede a la app y ve las mesas disponibles.
CU-02	Reservar mesa	Cliente	El cliente selecciona mesa, fecha, hora y confirma la reserva.
CU-03	Consultar carta	Cliente	El cliente revisa los platos y precios del restaurante.
CU-04	Acceder al panel admin	Administrador	El admin ingresa al panel con contraseña.
CU-05	Registrar mesa	Administrador	El admin crea un nuevo registro de mesa en el sistema.
CU-06	Gestionar horario	Administrador	El admin define o modifica el horario de atención.
CU-07	Ver reservas	Administrador	El admin consulta el listado de reservas.
CU-08	Cambiar estado de mesa	Administrador	El admin marca una mesa como activa o inactiva.

8.2 Caso de uso detallado: CU-02 – Reservar Mesa

A continuación, se desarrolla el caso de uso más importante del sistema de manera detallada.

Campo	Detalle
Código	CU-02
Nombre	Reservar Mesa
Actor principal	Cliente
Precondición	La aplicación está disponible. Existe al menos una mesa activa con horarios definidos.

Carlos Carrascal

Campo	Detalle
Flujo principal	1. El cliente abre la aplicación móvil. 2. El sistema muestra las mesas del restaurante. 3. El cliente selecciona una mesa. 4. El sistema muestra un calendario de fechas disponibles. 5. El cliente selecciona una fecha. 6. El sistema muestra los slots de hora disponibles y ocupados. 7. El cliente selecciona un slot disponible. 8. El sistema muestra un formulario para ingresar nombre, número de personas y (opcional) contacto. 9. El cliente ingresa sus datos y confirma la reserva. 10. El sistema guarda la reserva en Supabase y marca el slot como ocupado. 11. El sistema muestra un mensaje de confirmación con los datos de la reserva.
Flujo alternativo	En el paso 7: si el cliente selecciona un slot ya ocupado, el sistema muestra un mensaje de error y le pide seleccionar otro horario. En el paso 9: si el número de personas supera la capacidad de la mesa, el sistema informa el máximo permitido y solicita corregirlo.
Postcondición	La reserva queda registrada en la base de datos y el slot aparece como ocupado en la interfaz.

9. Propuesta de Arquitectura

La arquitectura de una aplicación define cómo se organizan y comunican sus componentes. Para este proyecto se propone una arquitectura de capas para una aplicación móvil multiplataforma construida con React Native y Expo, que se conecta directamente a Supabase.

9.1 Arquitectura en capas

Capa	Descripción
Capa de Presentación (Interfaz)	Pantallas y componentes de React Native (View, Text, FlatList, Pressable, TextInput). Es lo que el usuario ve y toca en su celular. Solo se encarga de mostrar datos y capturar eventos.
Capa de Navegación	Expo Router (o React Navigation) define las pestañas inferiores (Inicio, Carta, Admin) y la navegación en pila del flujo de reserva (reservar mesa → confirmación).
Capa de Lógica (servicios y hooks)	Archivos JavaScript con la lógica de negocio: generar los slots a partir de los horarios, validar el formulario, verificar la disponibilidad antes de reservar y construir las consultas a Supabase. El estado de las pantallas se maneja con hooks (useState, useEffect).
Capa de Datos (Supabase)	Supabase actúa como back-end de datos. La app usa el SDK @supabase/supabase-js para leer y escribir mesas, horarios, reservas y platos. No hay un servidor propio.

9.2 Flujo de comunicación

El flujo de datos entre las capas es el siguiente:

1. El usuario abre la app en su celular (con Expo Go durante el desarrollo, o con el APK instalado).
2. La pantalla ejecuta una función de la capa de servicios (por ejemplo, obtenerMesas()).
3. El SDK de Supabase envía la solicitud por HTTPS y Supabase consulta su base de datos.
4. Supabase devuelve los datos en formato JSON.
5. La app guarda el resultado en el estado del componente y la interfaz se vuelve a renderizar automáticamente.

> 💡 **¿Por qué React Native y Expo?**
> React Native permite escribir una sola base de código en JavaScript para Android e iOS. Expo simplifica la configuración: se prueba la app en el celular escaneando un código QR con Expo Go, sin instalar Android Studio ni Xcode, y se genera el APK con EAS Build. Para un proyecto académico es la forma más rápida de llegar a una app móvil funcional.

9.3 Stack tecnológico

Tecnología	Uso
React Native	Construcción de la interfaz móvil con componentes nativos para Android e iOS.
Expo (SDK)	Herramientas y librerías que facilitan el desarrollo, la prueba y la compilación de la app.
Expo Router / React Navigation	Navegación por pestañas y entre pantallas.
JavaScript (ES6+) o TypeScript	Lógica de la app, hooks y llamadas a la API.
Supabase (@supabase/supabase-js)	Base de datos como servicio. Se accede mediante su SDK.
AsyncStorage	Almacenamiento local en el celular (por ejemplo, para recordar la sesión del administrador).
GitHub	Control de versiones y repositorio del código fuente.
Expo Go y EAS Build	Prueba en el celular y generación del APK/IPA para distribuir la app.
Figma / Balsamiq / papel	Elaboración de los mockups de baja fidelidad.

10. Modelo de Datos

El modelo de datos describe cómo se organizará y almacenará la información en Supabase. Los datos se organizan en colecciones de registros con campos definidos.

> 📌 **Base de datos relacional vs no relacional**
> En una base de datos relacional (como MySQL o PostgreSQL), los datos se organizan en tablas con filas y columnas. En una base de datos no relacional, los datos se almacenan como colecciones de objetos JSON, lo que da más flexibilidad para datos con estructura variable. Supabase está construido sobre PostgreSQL; para este proyecto se usará su interfaz de tablas/colecciones y se podrán guardar campos en formato JSON cuando haga falta.

10.1 Colección: mesas

Almacena la información de cada mesa del restaurante.

Campo	Descripción
id	Identificador único de la mesa (generado automáticamente por Supabase).
nombre	Nombre o número de la mesa. Ej: 'Mesa 3'.
zona	Ubicación de la mesa. Ej: 'terraza' o 'salon'.
capacidad	Número máximo de personas. Ej: 4.
activa	Boolean que indica si la mesa está disponible para reservas. Ej: true/false.
foto_url	URL opcional de la foto de la mesa o de su zona para mostrar en la interfaz.
creado_en	Timestamp de cuando fue registrada la mesa en el sistema.

10.2 Colección: horarios

Define el horario de atención del restaurante por día de la semana.

Campo	Descripción
id	Identificador único del registro de horario.
dia_semana	Día de la semana. Ej: 'lunes', 'martes', etc.
jornada	Franja del día: 'almuerzo' (12:30 p. m. – 2:30 p. m.) o 'noche' (5:00 p. m. – 11:00 p. m.). Un mismo día puede tener dos registros.

Campo	Descripción
hora_inicio	Hora de apertura de la jornada. Ej: '12:30'.
hora_fin	Hora de cierre de la jornada. Ej: '14:30'.
duracion_slot	Duración en minutos de cada reserva. Ej: 60.
activo	Boolean que indica si este horario está vigente.

10.3 Colección: reservas

Registra cada reserva realizada por un cliente.

Campo	Descripción
id	Identificador único de la reserva (generado automáticamente).
mesa_id	Referencia a la mesa reservada.
cliente_nombre	Nombre del cliente que realizó la reserva.
cliente_contacto	Número de teléfono u otro medio de contacto (opcional).
num_personas	Cantidad de personas de la reserva. No puede superar la capacidad de la mesa.
fecha	Fecha de la reserva. Ej: '2026-11-15'.
hora_inicio	Hora de inicio de la reserva. Ej: '19:00'.
hora_fin	Hora estimada de fin. Ej: '20:00'.
estado	Estado de la reserva: 'confirmada', 'cancelada' o 'completada'.
creado_en	Timestamp de cuando se realizó la reserva.

10.4 Colección: platos

Representa los platos de la carta digital del restaurante.

Campo	Descripción
id	Identificador único del plato.
nombre	Nombre del plato. Ej: 'Pizza Konys Especial'.
categoria	Categoría: 'pizzas', 'entradas', 'postres', 'bebidas'.
descripcion	Descripción breve de los ingredientes.
precio	Precio en pesos colombianos (COP).
disponible	Boolean que indica si el plato se muestra en la carta.

11. Diseño de Interfaz

Esta sección describe la estructura de las pantallas de la aplicación móvil y presenta los mockups de baja fidelidad que guían el diseño visual durante el desarrollo. Se diseña con un enfoque 'mobile first': todo se piensa para pantallas verticales, interacción táctil y uso con una sola mano.

11.1 Navegación de la aplicación

- Barra de pestañas inferior con tres secciones: **Inicio** (mesas y reservas), **Carta** y **Admin**.
- Navegación en pila para el flujo de reserva: Inicio → Reservar mesa → Confirmación, con botón de regreso.
- El acceso a Admin solicita un PIN o contraseña simple antes de mostrar las pantallas de administración.
- Dentro de Admin hay pestañas de Mesas, Horarios y Reservas; la gestión de la Carta se abre desde un botón en la barra superior del panel.

11.2 Pantallas del cliente

- Inicio: encabezado con el nombre y el eslogan de Konys Pizza, los horarios de atención (12:30–2:30 p. m. y 5:00–11:00 p. m.), un botón 'Reservar mesa' y una lista (FlatList) de tarjetas de mesa con foto, nombre, zona, capacidad y estado.
- Reservar mesa: selector horizontal de fechas, grilla de horas dividida en 'Almuerzo' y 'Noche' (libres en blanco, ocupadas en gris, seleccionada en oscuro), formulario con nombre, teléfono y número de personas, resumen de la reserva y botón 'Confirmar reserva'.
- Confirmación: mensaje de éxito con los datos de la reserva y botón para volver al inicio.
- Carta: chips de categorías (Pizzas, Entradas, Postres, Bebidas) y lista de platos con foto, descripción y precio.

11.3 Pantallas del administrador

- Acceso: campo de PIN o contraseña y botón 'Ingresar'; si es incorrecto, se muestra un aviso.
- Mesas: lista de mesas con interruptor Activa/Inactiva, botones Editar y Borrar, y un botón flotante (+) que abre el formulario 'Nueva mesa'.
- Horarios: lista de los siete días de la semana, con las jornadas de almuerzo y noche editables.
- Reservas: filtros rápidos (Hoy, Mañana, Todas) y tarjetas con cliente, mesa, hora, personas y estado.
- Carta (admin): lista de platos con acciones para agregar, editar u ocultar.

11.4 Paleta de colores y tipografía (sugerencia)

Elemento	Descripción
Color primario	Azul petróleo oscuro (#0F2B3D) – coincide con el fondo del logo de Konys.
Color de acento	Naranja (#F28C28) – color del logo; resalta botones, chips activos y títulos.
Color de fondo	Blanco o gris muy claro para el contenido; el modo oscuro puede ser una mejora futura.
Tipografía	Fuente del sistema o una fuente de Google Fonts cargada con expo-font (por ejemplo 'Poppins').
Iconografía	Íconos de @expo/vector-icons (Ionicons, MaterialCommunityIcons) para pestañas y botones.
Tamaño táctil	Botones y slots de al menos 44 x 44 puntos para poder tocarlos cómodamente con el dedo.

11.5 Mockups de baja fidelidad

Un mockup de baja fidelidad (wireframe) es un boceto simple, en escala de grises y sin detalles visuales, que muestra la distribución de los elementos en cada pantalla. Su objetivo es validar la estructura y el flujo antes de escribir código. A continuación se presentan los mockups de referencia de la aplicación móvil.

> 📌 **Entregable del estudiante**
> Cada grupo debe elaborar sus propios mockups de baja fidelidad (en papel, Figma, Balsamiq, Excalidraw o similar) para, como mínimo, estas pantallas: inicio, reservar mesa, confirmación, carta, acceso de administrador y una pantalla de administración. Deben incluirse como imágenes en el documento final y subirse a la carpeta /docs del repositorio de GitHub.

12.2 Flujo del administrador

1. El administrador abre la pestaña 'Admin'.
2. Ingresa el PIN o la contraseña de acceso al panel.
3. Desde las pestañas, selecciona la sección que desea gestionar.
4. Para registrar una mesa: completa el formulario y guarda. Supabase almacena el nuevo registro.
5. Para gestionar horarios: edita el horario por día y guarda los cambios.
6. Para ver reservas: aplica filtros y visualiza la tabla con los registros.
7. Para cambiar el estado de una mesa: activa o desactiva el interruptor. El cambio se refleja en tiempo real para los clientes.

> ⚠️ **Condición de carrera (Race Condition)**
> Existe un riesgo técnico llamado 'condición de carrera': dos clientes reservan el mismo slot al mismo tiempo. Para mitigarlo, en el paso 10 del flujo de cliente se debe realizar una verificación en Supabase antes de confirmar la reserva. Si el slot ya fue tomado por otro cliente, se informa y se pide elegir otro. Aunque en producción se usarían transacciones atómicas, esta verificación doble es suficiente para el alcance del proyecto.

13. Plan de Desarrollo

El plan de desarrollo divide el proyecto en etapas manejables. Este tipo de planificación incremental permite entregar valor progresivamente y detectar problemas tempranamente. Cada etapa debe cerrar con al menos un commit en GitHub.

Etapa	Actividades
Etapa 1 – Análisis y diseño (~1 semana)	Leer y comprender este documento. Crear los mockups de baja fidelidad en papel o en Figma y el mapa de navegación. Definir la estructura final de las colecciones en Supabase. Crear el repositorio en GitHub (commit 1 y 2).
Etapa 2 – Configuración del entorno (~2 días)	Instalar Node.js y crear el proyecto con npx create-expo-app. Instalar Expo Go en el celular y probar la app base con npx expo start. Crear el proyecto en Supabase y las colecciones (mesas, horarios, reservas, platos). Configurar las variables de entorno.
Etapa 3 – Módulo cliente (~1–2 semanas)	Crear la navegación por pestañas y la pantalla de inicio con la lista de mesas. Implementar la pantalla de reserva: selector de fecha, grilla de horas por jornada y formulario. Conectar con Supabase para leer mesas y escribir reservas. Construir la pantalla de carta (commit 3, 4 y 5).
Etapa 4 – Módulo administrador (~1–2 semanas)	Implementar el acceso por PIN. Desarrollar las pantallas de mesas, horarios, reservas y carta con operaciones CRUD sobre Supabase (commit 6).
Etapa 5 – Pruebas (~3–5 días)	Probar el flujo completo en Expo Go y en un emulador (Android/iOS). Verificar distintos tamaños de pantalla y el comportamiento sin internet. Probar casos de error: slot ocupado, campos vacíos, personas por encima de la capacidad, sin mesas activas. Corregir errores (commit 7).
Etapa 6 – Compilación, video y entrega (~1–2 días)	Generar el APK con EAS Build (perfil preview) o publicar el proyecto en Expo para compartirlo por QR. Completar el README con instrucciones y enlaces. Grabar el video explicativo y entregar el repositorio, el APK o enlace de Expo y el video (commit 8).

> 💡 **Consejo de gestión del tiempo**
> No subestimes las etapas 1 y 2. Muchos estudiantes saltan directamente a escribir código sin planificar, lo que resulta en pantallas duplicadas o estructuras de datos incorrectas que obligan a rehacer todo desde cero. Prueba la app en un celular real desde el primer día.

14. Consideraciones de Despliegue

14.1 Configuración de Supabase

- Crear una cuenta gratuita en supabase.com.
- Crear un nuevo proyecto y anotar la Project URL y la anon/public key.
- Crear las colecciones descritas en el modelo de datos.
- Configurar las reglas de seguridad (Row Level Security - RLS) para controlar quién puede leer y escribir datos. Para producción, se recomienda restringir la escritura desde el cliente público.

14.2 Configuración de GitHub

- Crear un repositorio público (o privado con acceso al docente) con un nombre descriptivo, ej. konys-reservas-app.
- Incluir un archivo README.md con la descripción del proyecto, instrucciones para ejecutarlo y créditos.
- Usar buenas prácticas de commits: mensajes descriptivos y commits atómicos por funcionalidad.
- Incluir un .gitignore (node_modules, .expo, .env) para no subir archivos pesados ni sensibles.

14.3 Configuración de Expo

- Instalar Node.js (versión LTS) y una cuenta gratuita en expo.dev.
- Instalar la app **Expo Go** en el celular (Android o iOS) y conectarlo a la misma red Wi-Fi del computador.
- Crear el proyecto y ejecutarlo; al escanear el código QR la app se abre en el celular y se actualiza al guardar cambios.
- Guardar las credenciales de Supabase en un archivo .env con las variables EXPO_PUBLIC_SUPABASE_URL y EXPO_PUBLIC_SUPABASE_ANON_KEY, e incluir un .env.example sin valores reales.
- Para entregar la app instalable, usar **EAS Build** con un perfil 'preview' configurado para generar un APK de Android, que se descarga desde expo.dev e instala en el celular.

```
# Crear y ejecutar el proyecto
npx create-expo-app@latest konys-reservas-app
cd konys-reservas-app
npx expo install @supabase/supabase-js react-native-url-polyfill
@react-native-async-storage/async-storage
npx expo start

# Generar el APK (requiere cuenta de Expo)
npm install -g eas-cli
eas login
eas build:configure
eas build -p android --profile preview
```

> ⚠️ **Seguridad con las credenciales**
> Las variables que empiezan por EXPO_PUBLIC_ quedan incluidas dentro de la aplicación compilada, así que cualquier persona podría extraerlas. Por eso solo se debe usar la clave anon de Supabase, nunca la service_role, y proteger los datos con reglas de Row Level Security. No subas el archivo .env al repositorio.

15. Entrega en GitHub

El código fuente y la documentación del proyecto deben entregarse en un repositorio de GitHub. El historial de commits es la evidencia del proceso de desarrollo: muestra cómo evolucionó el proyecto, quién trabajó en qué y cuándo.

15.1 Requisitos de la entrega

- **Mínimo 5 commits** con mensajes descriptivos, realizados en momentos distintos del desarrollo (no se aceptan todos los commits en un solo día ni un único commit con todo el proyecto).
- Si el trabajo es en pareja o grupo, **cada integrante debe tener commits** a su nombre.
- Repositorio con nombre descriptivo (ej: konys-reservas-app), público o privado con acceso al docente.
- Archivo **README.md** con: descripción del proyecto, tecnologías, instrucciones para ejecutarlo (npm install y npx expo start), enlace del APK o del proyecto en Expo y enlace del video.
- Archivos **.gitignore** y **.env.example**; ningún secreto (claves de Supabase) dentro del código.
- Carpeta **/docs** con este documento de proyecto y las imágenes de los mockups.
- **Evidencia:** incluye la captura de la pestaña 'Commits' de GitHub (o de git log --oneline) donde se vean al menos 5 commits con sus fechas.

Xdxxxx vfghfh

15.2 Estructura sugerida del repositorio

```
konys-reservas-app/
├── app/                          # Pantallas (Expo Router)
│   ├── _layout.js
│   ├── (tabs)/
│   │   ├── index.js              # Inicio (mesas)
│   │   ├── carta.js
│   │   └── admin.js
│   ├── reservar/[mesaId].js      # Reservar mesa
│   └── confirmacion.js
├── components/                   # MesaCard, SlotButton, PlatoItem
├── services/
│   ├── supabase.js               # Cliente de Supabase
│   └── reservas.js               # Lógica de reservas y slots
├── constants/colors.js
├── assets/
├── docs/                         # Documento y mockups
├── .env.example
├── .gitignore
├── app.json
├── eas.json
└── README.md
```

15.3 Plan de commits sugerido

La siguiente tabla propone ocho commits, alineados con las etapas del plan de desarrollo. Son un mínimo de referencia: se pueden hacer más commits, pero nunca menos de cinco.

#	Mensaje del commit	Contenido
1	chore: proyecto Expo inicial y README	Proyecto creado con create-expo-app, .gitignore, .env.example y README con la descripción de Konys Pizza.
2	docs: agregar documento de proyecto y mockups	Documento en /docs y wireframes de baja fidelidad.
3	feat: navegación por pestañas y pantalla de inicio con mesas	Expo Router, tabs y lista de tarjetas de mesa.
4	feat: pantalla de reserva con fecha, jornada y formulario	Selector de fecha, grilla de horas, validaciones y confirmación.
5	feat: integración con Supabase	Conexión, lectura de mesas y platos, escritura de reservas.
6	feat: módulo de administración (mesas, horarios, reservas)	Pantallas de admin con acceso por PIN y CRUD.

#	Mensaje del commit	Contenido
7	fix: validación de doble reserva y ajustes en Android e iOS	Corrección de errores detectados en las pruebas.
8	chore: configuración de EAS Build y enlaces en el README	eas.json, APK generado, enlace del video.

15.4 Comandos básicos

```
git init
git add .
git commit -m "chore: proyecto Expo inicial y README"
git branch -M main
git remote add origin https://github.com/<usuario>/konys-reservas-app.git
git push -u origin main

# Para cada avance posterior:
git add .
git commit -m "feat: pantalla de reserva con fecha, jornada y formulario"
git push

# Verificar el historial (mínimo 5 commits):
git log --oneline
```

16. Video Explicativo

Además del código, cada grupo debe grabar un video en el que explique el proyecto con sus propias palabras. El video permite verificar que el estudiante comprende lo que construyó y no solo que la app funciona.

16.1 Requisitos del video

- Duración entre **5 y 8 minutos**.
- Deben participar **todos los integrantes** del grupo, explicando su parte del trabajo (se recomienda mostrar la voz y, si es posible, el rostro).
- La demostración debe hacerse en la app corriendo en un **celular real o en un emulador**, grabando la pantalla del dispositivo, con audio claro.
- **Consejo:** ensaya antes de grabar y prepara los datos de prueba en Supabase; un video corto, ordenado y con la app funcionando vale más que uno largo con improvisaciones.
- Publicado en YouTube (no listado) o Google Drive con acceso para el docente, y con el enlace incluido en el README y en el documento final.

16.2 Guion sugerido

Minuto	Contenido
0:00 – 0:45	Presentación del grupo y del proyecto: qué es Konys Pizza y qué problema resuelve la app de reservas.
0:45 – 2:00	Explicación del diseño: requisitos principales, casos de uso, mockups de baja fidelidad y mapa de navegación. Mostrar cómo los mockups se transformaron en las pantallas finales.
2:00 – 4:00	Demostración del cliente en el celular: ver mesas, elegir fecha y jornada, reservar, ver la confirmación y consultar la carta. Probar también un caso de error (slot ocupado).
4:00 – 5:15	Demostración del administrador: acceso con PIN, registro de una mesa, edición de horarios, consulta de reservas y cambio de estado.
5:15 – 6:30	Explicación técnica: estructura de carpetas de React Native, uso de Expo, conexión con Supabase (mostrar las colecciones con datos reales) y cómo se generó el APK.
6:30 – 7:30	Recorrido por GitHub: repositorio, README e historial con **mínimo 5 commits**; explicar qué hizo cada integrante.
7:30 – 8:00	Conclusiones: dificultades encontradas, aprendizajes y posibles mejoras futuras.

17. Entregables del Proyecto

Al finalizar el proyecto, cada grupo debe presentar los siguientes entregables:

Entregable	Descripción
Documento de proyecto	Este documento completado, incluyendo los mockups de baja fidelidad elaborados por el grupo.
Mockups de baja fidelidad	Imágenes de las pantallas principales (cliente y administrador), también en la carpeta /docs del repositorio.
Repositorio de GitHub	Enlace al repositorio con **mínimo 5 commits**, README, .gitignore y estructura ordenada.
Aplicación móvil	APK instalable generado con EAS Build, o enlace/código QR del proyecto en Expo para abrirlo con Expo Go, conectado a Supabase.
Video explicativo	Enlace al video de 5 a 8 minutos con la explicación y demostración del proyecto.

17.1 Mejoras futuras

Las siguientes funcionalidades están fuera del alcance del proyecto actual pero representan una evolución natural de la aplicación para versiones posteriores.

Mejora	Descripción
Autenticación de clientes	Implementar registro e inicio de sesión (por ejemplo con Supabase Auth) para que los clientes vean su historial, modifiquen o cancelen reservas.
Notificaciones push	Recordatorios de reserva al cliente y avisos de nuevas reservas al administrador con expo-notifications.
Pedidos a domicilio	Integrar el servicio de domicilios de Konys Pizza con un carrito de compras y seguimiento del pedido.
Sistema de pagos	Integrar una pasarela de pago para permitir un anticipo de la reserva en fechas de alta demanda.
Publicación en tiendas	Publicar la app en Google Play y App Store usando EAS Submit.
Panel de estadísticas	Gráficas de demanda por día y jornada, mesa más solicitada y tasa de no-show.
Reservas para eventos	Gestionar solicitudes de eventos, ya que Konys Pizza llega a eventos con su servicio.
Modo oscuro	Tema claro/oscuro según la configuración del celular.

> 📌 **Reflexión final para el estudiante**
> Este proyecto cubre todas las fases de la ingeniería de software: análisis, diseño, desarrollo, pruebas y despliegue. Al completarlo, habrás aplicado conceptos clave como: levantamiento de requisitos, modelado de datos, prototipado con mockups, desarrollo móvil multiplataforma con React Native y Expo, control de versiones con Git y consumo de APIs externas. Cada dificultad que encuentres en el camino es una oportunidad de aprendizaje. ¡Éxito con Konys Pizza!